```
"""
memory_store.py
────────────────

Persistent Bayesian memory for the scaffold agent.
Stores per-user belief state, interaction history, and bias fingerprint.
Backend: JSON file (swap for SQLite/Redis trivially — see bottom).
"""

import json
import os
import time
from pathlib import Path
from typing import Optional

DEFAULT_PATH = Path("./scaffold_memory.json")

# ── Default user state ──────────────────────────────────────────────────

def _default_user() -> dict:
    return {
        "created_at": time.time(),
        "updated_at": time.time(),
        "rounds": 0,
        # Posteriors
        "human_prior": 0.50,   # P(human knows what they want)
        "ai_prior":    0.50,   # P(AI can satisfy without clarification)
        # Bias fingerprint — cumulative hit counts
        "bias_counts": {
            "anchoring":          0,
            "vague_intent":       0,
            "confirmation_bias":  0,
            "dunning_kruger":     0,
            "framing_effect":     0,
            "clear_intent":       0,
            "unknown":            0,
        },
        # Outcome history
        "outcome_counts": {
            "right_exact":    0,
            "right_partial":  0,
            "right_lucky":    0,
            "clarify":        0,
            "wrong_close":    0,
            "wrong_badly":    0,
```

```
        },
        # Rolling window: last N interactions (for trend detection)
        "recent": [],          # list of {bias, outcome, timestamp}
        "recent_window": 10,
        # Agent personality drift (0=fully compliant, 1=fully assertive)
        "agent_assertiveness": 0.20,
        # Session metadata
        "sessions": 0,
        "last_session": None,
    }


# ── Store class ──────────────────────────────────────────────────────

class MemoryStore:
    """
    Thread-unsafe single-file JSON store.
    Fine for a single-user local agent; swap backend for multi-user.
    """

    def __init__(self, path: Path = DEFAULT_PATH):
        self.path = Path(path)
        self._data: dict = {}
        self._load()

    # ── I/O ──────────────────────────────────────────────────────────

    def _load(self):
        if self.path.exists():
            with open(self.path, "r") as f:
                self._data = json.load(f)
        else:
            self._data = {}

    def _save(self):
        self.path.parent.mkdir(parents=True, exist_ok=True)
        with open(self.path, "w") as f:
            json.dump(self._data, f, indent=2)

    # ── User access ──────────────────────────────────────────────────

    def get_user(self, user_id: str = "default") -> dict:
        if user_id not in self._data:
            self._data[user_id] = _default_user()
            self._save()
        return self._data[user_id]
```

```
def save_user(self, user_id: str, state: dict):
    state["updated_at"] = time.time()
    self._data[user_id] = state
    self._save()

# ── Belief updates ──────────────────────────────────────────────────────

def update_priors(
    self,
    user_id: str,
    new_human_prior: float,
    new_ai_prior: float,
    bias: str,
    outcome: str,
):
    u = self.get_user(user_id)
    u["human_prior"]  = round(new_human_prior, 4)
    u["ai_prior"]     = round(new_ai_prior, 4)
    u["rounds"]      += 1

    # Bias count
    b = bias.lower().replace(" ", "_")
    if b in u["bias_counts"]:
        u["bias_counts"][b] += 1
    else:
        u["bias_counts"]["unknown"] += 1

    # Outcome count
    if outcome in u["outcome_counts"]:
        u["outcome_counts"][outcome] += 1

    # Rolling window
    u["recent"].append({
        "bias": b,
        "outcome": outcome,
        "timestamp": time.time(),
    })
    if len(u["recent"]) > u["recent_window"]:
        u["recent"] = u["recent"][-u["recent_window"]:]

    # Assertiveness drift: AI gets bolder after repeated wrong/clarify
    wrong_rate = self._recent_wrong_rate(u)
    u["agent_assertiveness"] = round(
        min(0.90, u["agent_assertiveness"] + wrong_rate * 0.05), 4
    )

    self.save_user(user_id, u)
```

```
def _recent_wrong_rate(self, u: dict) -> float:
    if not u["recent"]:
        return 0.0
    wrong = sum(
        1 for r in u["recent"]
        if r["outcome"].startswith("wrong") or r["outcome"] == "clarify"
    )
    return wrong / len(u["recent"])

# ── Session tracking ──────────────────────────────────────────────

def start_session(self, user_id: str):
    u = self.get_user(user_id)
    u["sessions"] += 1
    u["last_session"] = time.time()
    self.save_user(user_id, u)

# ── Snapshot for HTML visualiser ──────────────────────────────────

def export_snapshot(self, user_id: str = "default") -> dict:
    """Returns a JSON-serialisable snapshot for the HTML tool."""
    u = self.get_user(user_id)
    total = u["rounds"] or 1
    oc    = u["outcome_counts"]
    hits  = oc["right_exact"] + oc["right_partial"] + oc["right_lucky"]
    return {
        "user_id":            user_id,
        "rounds":             u["rounds"],
        "human_prior":        u["human_prior"],
        "ai_prior":           u["ai_prior"],
        "agent_assertiveness": u["agent_assertiveness"],
        "hit_rate":           round(hits / total, 3),
        "clarify_rate":       round(oc["clarify"] / total, 3),
        "wrong_rate":         round((oc["wrong_close"] + oc["wrong_badly"]) /
        "dominant_bias":      max(u["bias_counts"], key=u["bias_counts"].get)
        "bias_counts":        u["bias_counts"],
        "outcome_counts":     u["outcome_counts"],
        "recent":             u["recent"],
        "sessions":           u["sessions"],
    }

# ── Reset ─────────────────────────────────────────────────────────

def reset_user(self, user_id: str = "default"):
    self._data[user_id] = _default_user()
    self._save()
```

```
# ── SQLite shim (drop-in swap) ──────────────────────────────────────
# To use SQLite instead, subclass MemoryStore and override _load/_save/_data
# access with sqlite3 calls. The rest of the API stays identical.
```